EN3160 Assignment 2 - Fitting and Allignment

Name: Perera U. I. Himal

Index No: 230486A

GitHub: https://github.com/Isira-Himal-Perera/EN3160-Image-Processing-and-Computer-Vision/tree/main/assignment-02

Question 1 - Blob Detection

In [ ]:
```
import cv2 as cv
import matplotlib.pyplot as plt
import numpy as np

# 1. Load the image as specified in the assignment[cite: 1]
im = cv.imread(
    'images/the_berry_farms_sunflower_field.jpeg', cv.IMREAD_REDUCED_COLOR_4
)

# Keep the original color image for drawing, and convert to grayscale for LoG detec
im_gray = cv.cvtColor(im, cv.COLOR_BGR2GRAY)
im_float = im_gray.astype(np.float32) / 255.0

# Define sigma range
# min_sigma = 2.0
# max_sigma = 17.0
# num_sigmas = 18
# sigmas = np.linspace(min_sigma, max_sigma, num_sigmas)

sigmas = np.arange(2, 20)
# Initialize scale-space volume (H, W, num_sigmas)
h, w = im_float.shape
scale_space = np.zeros((h, w, len(sigmas)), dtype=np.float32)

# Build scale-space volume using scale-normalized Laplacian of Gaussian
for i, sigma in enumerate(sigmas):
    ksize = int(2 * np.ceil(3 * sigma) + 1)
    # Ensure kernel size is odd
    if ksize % 2 == 0:
        ksize += 1
    blurred = cv.GaussianBlur(im_float, (ksize, ksize), sigmaX=sigma, sigmaY=sigma)
    laplacian = cv.Laplacian(blurred, cv.CV_32F, ksize=3)
    scale_space[:, :, i] = (sigma**2) * np.abs(laplacian)

# Extract local maxima (blobs) across 3D scale-space (y, x, scale)
blobs = []
threshold = 0.7 * np.max(scale_space)  # Adjusted relative threshold
for i, sigma in enumerate(sigmas):
    slice_curr = scale_space[:, :, i]
```

```
    # Find local maxima in the 2D spatial plane using OpenCV morphology dilation
    ksize_morph = int(2 * round(2 * sigma) + 1)
    if ksize_morph % 2 == 0:
        ksize_morph += 1
    kernel = cv.getStructuringElement(cv.MORPH_RECT, (ksize_morph, ksize_morph))
    local_max_xy = cv.dilate(slice_curr, kernel)

    # Get neighboring scales for 3D scale-space comparison
    upper = (scale_space[:, :, i + 1] if i < len(sigmas) - 1 else np.zeros_like(sli
    lower = scale_space[:, :, i - 1] if i > 0 else np.zeros_like(slice_curr)

    is_max = (
        (slice_curr == local_max_xy)
        & (slice_curr >= upper)
        & (slice_curr >= lower)
        & (slice_curr > threshold)
    )

    y_idxs, x_idxs = np.nonzero(is_max)
    for y, x in zip(y_idxs, x_idxs):
        blobs.append((y, x, sigma))

# Draw circles on a copy of the original color image (OpenCV uses BGR natively)
output_im_bgr = im.copy()

for y, x, sigma in blobs:
    # Radius formula for LoG blob detection: r = sigma * sqrt(2)
    radius = int(sigma * np.sqrt(2))
    cv.circle(output_im_bgr, (x, y), radius, (0, 0, 255), 1)  # Yellow circle (BGR)

# Convert the final output image to RGB format as requested
output_im_rgb = cv.cvtColor(output_im_bgr, cv.COLOR_BGR2RGB)

# Display using matplotlib
plt.figure(figsize=(12, 8))
plt.imshow(output_im_rgb)
plt.axis('off')
plt.title(f'Detected Sunflowers ({len(blobs)} blobs found)')
plt.show()
```



Question 2 - RANSAC

In [2]:
```
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

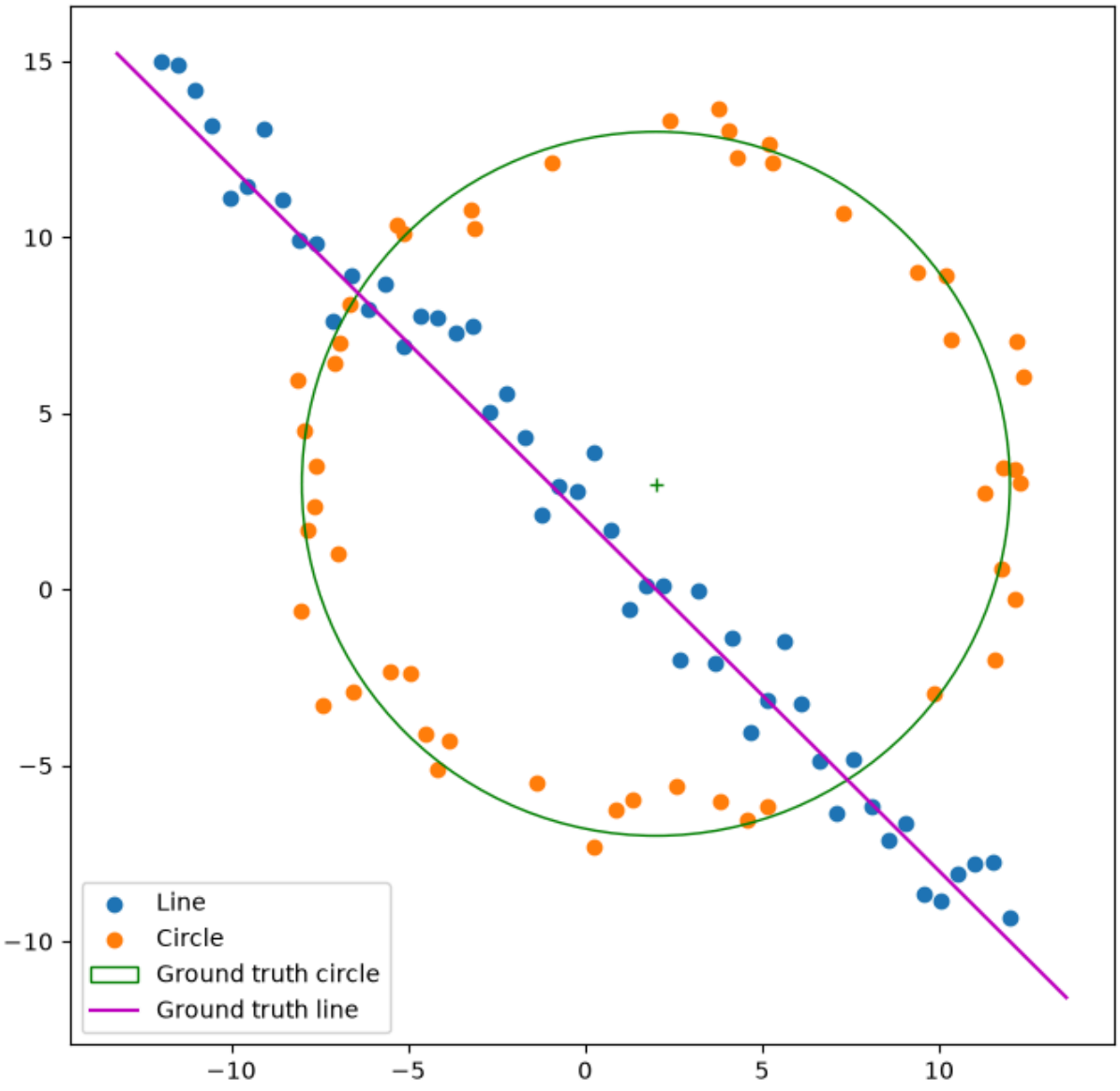
np.random.seed(486)
N = 100
half_n = N // 2
r = 10
x0_gt, y0_gt = 2, 3  # Center
s = r / 16
t = np.random.uniform(0, 2 * np.pi, half_n)
n = s * np.random.randn(half_n)
x, y = x0_gt + (r + n) * np.cos(t), y0_gt + (r + n) * np.sin(t)
X_circ = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))
```

```
s = 1.0
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
y = m * x + b + s * np.random.randn(half_n)
X_line = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))

X = np.vstack((X_circ, X_line))  # All points
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(X_line[:, 0], X_line[:, 1], label='Line')
ax.scatter(X_circ[:, 0], X_circ[:, 1], label='Circle')

circle_gt = plt.Circle((x0_gt, y0_gt), r, color='g', fill=False, label='Ground trut
ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, '+', color='g')

x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m * x_ + b
plt.plot(x_, y_, color='m', label='Ground truth line')
plt.legend()
plt.show()
```



(a) Generating the Line

In [3]:

```python
def fit_line_tls(points):
    # Compute the mean of the points
    mean = np.mean(points, axis=0)
    # Center the points by subtracting the mean
    U = points - mean
    # Compute the scatter/covariance matrix (transpose dot product)
    UtU = np.dot(U.T, U)
    # Perform eigenvalue decomposition on the scatter matrix
    eigenvalues, eigenvectors = np.linalg.eig(UtU)
    # Find the eigenvector corresponding to the smallest eigenvalue (Total Least Sq
    min_idx = np.argmin(eigenvalues)
    normal = eigenvectors[:, min_idx]
    # Normalize the normal vector
    normal = normal / np.linalg.norm(normal)
    a, b = normal[0], normal[1]
    # Compute the line offset d using the mean point
    d = np.dot(normal, mean)
```

```
    return a, b, d

def ransac_line(X, num_iterations=1000, threshold=2.0, min_inliers=50):
    best_a, best_b, best_d = None, None, None
    best_inliers = []
    best_sample = None
    max_inliers_count = 0
    N_points = X.shape[0]

    for _ in range(num_iterations):
        # Randomly select two points to define a tentative line model
        sample_indices = np.random.choice(N_points, 2, replace=False)
        p1 = X[sample_indices[0]]
        p2 = X[sample_indices[1]]

        # Calculate the direction vector and its orthogonal normal vector
        dir_vec = p2 - p1
        normal = np.array([-dir_vec[1], dir_vec[0]])
        norm_val = np.linalg.norm(normal)
        # Skip iteration if the sampled points are too close to each other
        if norm_val < 1e-9:
            continue
        normal = normal / norm_val
        a, b = normal[0], normal[1]
        d = a * p1[0] + b * p1[1]

        # Compute perpendicular distances of all points to the candidate line
        distances = np.abs(np.dot(X, np.array([a, b])) - d)
        # Identify points within the distance threshold as inliers
        inlier_indices = np.where(distances < threshold)[0]

        # Update the best model if the current inlier count exceeds the previous ma
        if len(inlier_indices) > max_inliers_count:
            max_inliers_count = len(inlier_indices)
            best_inliers = inlier_indices
            best_a, best_b, best_d = a, b, d
            best_sample = (p1, p2)

    # If enough inliers are found, refine the line parameters using Total Least Squ
    if len(best_inliers) >= min_inliers:
        best_a, best_b, best_d = np.real(fit_line_tls(X[best_inliers]))

    return best_a, best_b, best_d, best_inliers, best_sample
```

(b) generating the circle

In [4]:

```
def fit_circle_3points(p1, p2, p3):
    """
    Fits a circle through 3 given points using algebraic circle fitting.
    """
    A_mat = np.array([
        [p1[0], p1[1], 1],
        [p2[0], p2[1], 1],
        [p3[0], p3[1], 1]
    ])
```

```
    b_mat = -np.array([
        p1[0]**2 + p1[1]**2,
        p2[0]**2 + p2[1]**2,
        p3[0]**2 + p3[1]**2
    ])
    try:
        params = np.linalg.solve(A_mat, b_mat)
    except np.linalg.LinAlgError:
        return None, None, None

    A, B, C = params
    x0 = -A / 2.0
    y0 = -B / 2.0
    r_sq = x0**2 + y0**2 - C
    if r_sq < 0:
        return None, None, None
    r = np.sqrt(r_sq)
    return x0, y0, r

def fit_circle_ls(points):
    """
    Refines circle parameters (center and radius) using least squares optimization.
    """
    x0_init = np.mean(points[:, 0])
    y0_init = np.mean(points[:, 1])
    r_init = np.mean(np.sqrt((points[:, 0] - x0_init)**2 + (points[:, 1] - y0_init)

    def objective(params):
        x0, y0, r = params
        distances = np.sqrt((points[:, 0] - x0)**2 + (points[:, 1] - y0)**2)
        return np.sum((distances - r)**2)

    res = minimize(objective, [x0_init, y0_init, r_init])
    return res.x[0], res.x[1], res.x[2]

def ransac_circle(X, num_iterations=1000, threshold=2.0, min_inliers=25):
    """
    Estimates a circle from noisy 2D data using the RANSAC algorithm.
    """
    best_x0, best_y0, best_r = None, None, None
    best_inliers = []
    best_sample = None
    max_inliers_count = 0
    N_points = X.shape[0]

    if N_points < 3:
        return None, None, None, [], None

    for _ in range(num_iterations):
        # Randomly select three points to define a tentative circle model
        sample_indices = np.random.choice(N_points, 3, replace=False)
        p1, p2, p3 = X[sample_indices[0]], X[sample_indices[1]], X[sample_indices[2

        x0, y0, r = fit_circle_3points(p1, p2, p3)
        if x0 is None:
            continue
```

```
        # Compute distances of all points to the candidate circle boundary
        distances = np.abs(np.sqrt((X[:, 0] - x0)**2 + (X[:, 1] - y0)**2) - r)
        inlier_indices = np.where(distances < threshold)[0]

        # Update the best model if the current inlier count exceeds the previous ma
        if len(inlier_indices) > max_inliers_count:
            max_inliers_count = len(inlier_indices)
            best_inliers = inlier_indices
            best_x0, best_y0, best_r = x0, y0, r
            best_sample = (p1, p2, p3)

    # If enough inliers are found, refine the circle parameters using least squares
    if len(best_inliers) >= min_inliers:
        best_x0, best_y0, best_r = np.real(fit_circle_ls(X[best_inliers]))

    return best_x0, best_y0, best_r, best_inliers, best_sample

# Define RANSAC thresholds for line and circle extraction pipeline
line_threshold = 1.5
circle_threshold = 1.25

# Step 1: Detect and fit line inliers from the dataset
a_est, b_est, d_est, line_inliers, line_sample = ransac_line(X, 1000, line_threshol

# Step 2: Remove line inliers to isolate remaining points for circle fitting
line_mask = np.zeros(X.shape[0], dtype=bool)
line_mask[line_inliers] = True
X_remaining = X[~line_mask]

# Step 3: Detect and fit circle parameters from the remaining points
x0_est, y0_est, r_est, circle_inliers_rel, circle_sample = ransac_circle(X_remainin

# Step 4: Map the circle inliers back to their corresponding indices in the origina
remaining_indices = np.where(~line_mask)[0]
circle_inliers = remaining_indices[circle_inliers_rel]
```

(c) plotting the visualisation

In [5]:

```
fig, ax = plt.subplots(1, 1, figsize=(9, 9))

# Plot all data points (including outliers)
ax.scatter(X[:, 0], X[:, 1], label='All Points', zorder=1)

if line_sample:
    # Plot detected line inliers
    ax.scatter(X[line_inliers, 0], X[line_inliers, 1], color='y', label='Line Inlie

    # Plot the best sample points used for line estimation
    line_sample_arr = np.array(line_sample)
    ax.scatter(line_sample_arr[:, 0], line_sample_arr[:, 1], color='r', marker='x',

if circle_sample:
    # Plot detected circle inliers
    ax.scatter(X[circle_inliers, 0], X[circle_inliers, 1], color='c', label='Circle
```

```
    # Plot the best sample points used for circle estimation
    circle_sample_arr = np.array(circle_sample)
    ax.scatter(circle_sample_arr[:, 0], circle_sample_arr[:, 1], color='r', zorder=

# Plot the ground truth circle and its center
circle_gt = plt.Circle((x0_gt, y0_gt), r, color='g', fill=False, label='Ground trut
ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, '+', color='g')

if circle_sample:
    # Plot the estimated RANSAC circle and its center
    best_circle_patch = plt.Circle((x0_est, y0_est), r_est, fill=False, color='m',
    ax.add_patch(best_circle_patch)
    ax.plot(x0_est, y0_est, color='m')

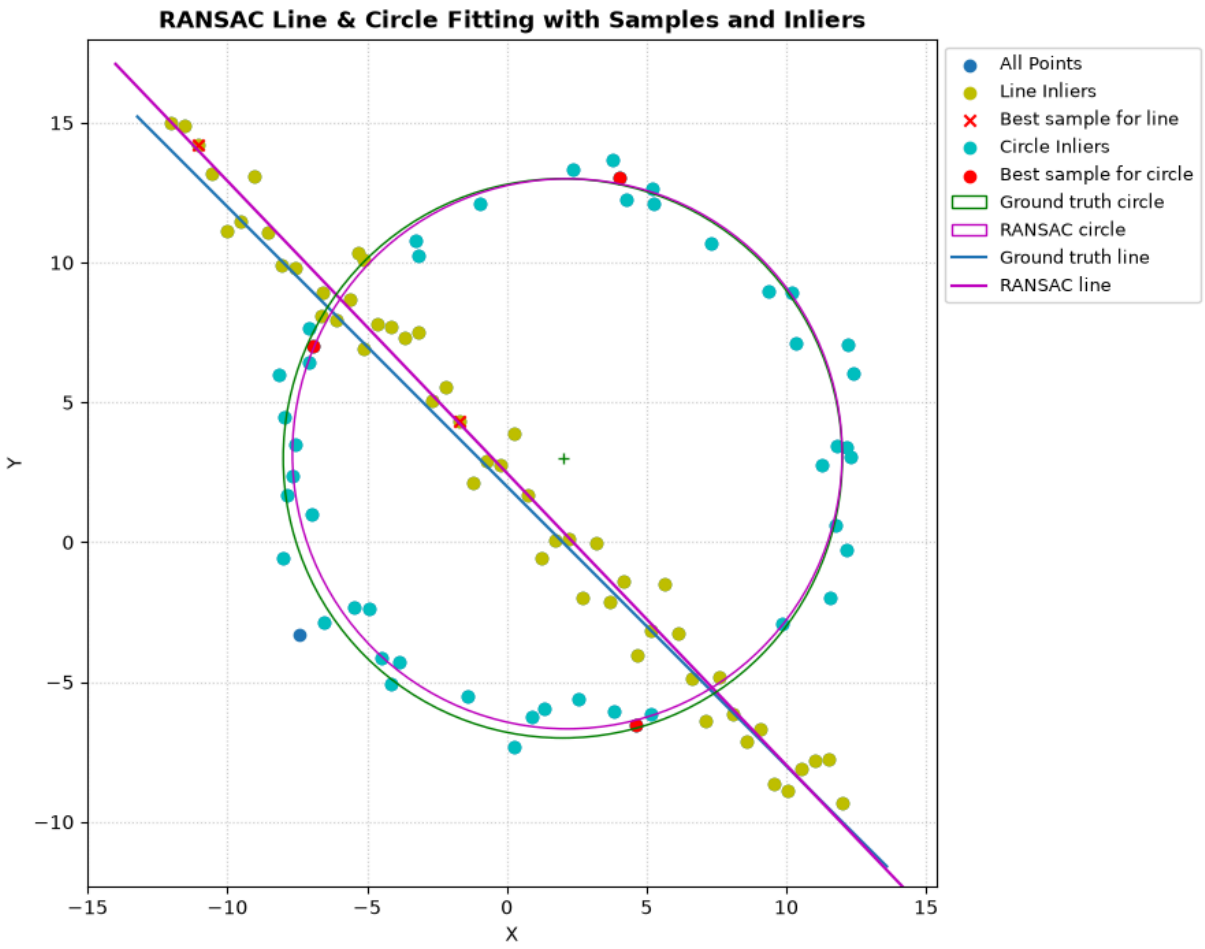
# Plot the ground truth line
plt.plot(x_, y_, label='Ground truth line')

if line_sample:
    # Calculate coordinate range to plot the estimated RANSAC line
    x_min, x_max = X[:, 0].min() - 2, X[:, 0].max() + 2
    x_vals = np.array([x_min, x_max])

    if abs(b_est) > 1e-6:   # If not a vertical line
        y_vals = (d_est - a_est * x_vals) / b_est
    else:                   # If a vertical line
        x_vals = np.array([d_est / a_est, d_est / a_est]) # Note: original logic ke
        y_vals = np.array([X[:, 1].min() - 2, X[:, 1].max() + 2])

    ax.plot(x_vals, y_vals, color='m', label='RANSAC line', zorder=3)

# Configure plot limits, aspect ratio, grid, and legend layout
ax.set_xlim(X[:, 0].min() - 3, X[:, 0].max() + 3)
ax.set_ylim(X[:, 1].min() - 3, X[:, 1].max() + 3)
ax.set_aspect('equal')
ax.grid(True, linestyle=':', alpha=0.6)
ax.legend(loc='upper left', bbox_to_anchor=(1, 1), fontsize=9)
plt.title("RANSAC Line & Circle Fitting with Samples and Inliers", fontsize=12, fon
plt.xlabel("X")
plt.ylabel("Y")
plt.tight_layout()
plt.show()
```



(d) Analysis: What will happen if we fit the circle first?

If we fit the circle first, the circle's RANSAC algorithm will incorrectly capture the segment of the line passing through the interior of the circle—including the intersection points—as circle inliers because they fall within the circle's radial error threshold.

Consequently, when these inliers are removed to fit the line next, the middle section of the line is deleted. This leaves fragmented, disconnected line segments, which prevents the line-fitting RANSAC algorithm from accurately recovering the true line.

Question (3) Homography

In [ ]:
```
im1 = cv.imread('images/wadham.jpg', cv.IMREAD_COLOR)
im2 = cv.imread('images/flag.png', cv.IMREAD_COLOR)

assert im1 is not None
assert im2 is not None

h, w = im2.shape[:2]  # Size of the overlay image
corners = []

# 2. Mouse callback function for interactive corner selection
def click_event(event, x, y, flags, params):
```

```
    if event == cv.EVENT_LBUTTONDOWN and len(corners) < 4:
        corners.append((x, y))
        # Draw a visual green dot on the display image for feedback
        cv.circle(image_display, (x, y), 5, (0, 255, 0), -1)
        cv.imshow('Select 4 Corners', image_display)

image_display = im1.copy()
cv.imshow('Select 4 Corners', image_display)
cv.setMouseCallback('Select 4 Corners', click_event)

print("Please click exactly 4 corners on the image window (press 'q' to abort)...")

# Keep window active until 4 points are selected or 'q' is pressed
while True:
    key = cv.waitKey(1) & 0xFF
    if len(corners) == 4 or key == ord('q'):
        break

cv.destroyAllWindows()

# 3. Error correction: Validate that exactly 4 corners were picked
if len(corners) != 4:
    raise ValueError(f"Error: Expected 4 points, but only {len(corners)} were selec

# 4. Compute homography with explicit float32 data types
pts_src = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]], dtype=np.float
pts_dst = np.array(corners, dtype=np.float32)

H, status = cv.findHomography(pts_src, pts_dst)
if H is None:
    raise RuntimeError("Error: Failed to compute homography matrix from the selecte

# 5. Warp and blend images
im_out = cv.warpPerspective(im2, H, (im1.shape[1], im1.shape[0]))
mask = cv.warpPerspective(np.full((h, w), 255, np.uint8), H, (im1.shape[1], im1.sha
mask3 = (mask > 0)[..., None]

alpha = 0.8  # Flag blending weight
blended = np.where(
    mask3,
    cv.addWeighted(im1, 1 - alpha, im_out, alpha, 0),
    im1.astype(np.uint8)
)

# 6. Display final blended result via OpenCV
cv.imshow('Blended Image', blended)
cv.waitKey(0)
cv.destroyAllWindows()

# 7. Plot results using Matplotlib
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
titles = ['Original Image', 'Overlay Image', 'Blended Image']
images = [im1, im2, blended]

for ax, im, title in zip(axes, images, titles):
    ax.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
```

```
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout()
plt.show()
```

```
Please click exactly 4 corners on the image window (press 'q' to abort)...
```



In [38]:

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

def run_homography_overlay(im1_path, im2_path, alpha=1.0, max_w=1280, max_h=720):
    """
    Loads base and overlay images, prompts the user to select 4 corners interactive
    computes the perspective homography, blends the images, and plots the results.
    """
    im1 = cv.imread(im1_path, cv.IMREAD_COLOR)
    im2 = cv.imread(im2_path, cv.IMREAD_COLOR)

    assert im1 is not None, f"Could not load base image from {im1_path}"
    assert im2 is not None, f"Could not load overlay image from {im2_path}"

    # Downsample im1 to HD (max 1280x720) if it's larger, preserving the aspect rat
    h_orig, w_orig = im1.shape[:2]
    if w_orig > max_w or h_orig > max_h:
        scale = min(max_w / w_orig, max_h / h_orig)
        new_w, new_h = int(w_orig * scale), int(h_orig * scale)
        im1 = cv.resize(im1, (new_w, new_h), interpolation=cv.INTER_AREA)
        print(f"Image downsampled from {w_orig}x{h_orig} to {new_w}x{new_h} to fit

    h, w = im2.shape[:2]  # Size of the overlay image
    corners = []

    # 2. Mouse callback function for interactive corner selection
    def click_event(event, x, y, flags, params):
        if event == cv.EVENT_LBUTTONDOWN and len(corners) < 4:
            corners.append((x, y))
            # Draw a visual green dot on the display image for feedback
            cv.circle(image_display, (x, y), 5, (0, 255, 0), -1)
            cv.imshow('Select 4 Corners', image_display)

    image_display = im1.copy()
    cv.imshow('Select 4 Corners', image_display)
    cv.setMouseCallback('Select 4 Corners', click_event)
```

```
    print("Please click exactly 4 corners on the image window (press 'q' to abort).

    # Keep window active until 4 points are selected or 'q' is pressed
    while True:
        key = cv.waitKey(1) & 0xFF
        if len(corners) == 4 or key == ord('q'):
            break

    cv.destroyAllWindows()

    # 3. Error correction: Validate that exactly 4 corners were picked
    if len(corners) != 4:
        raise ValueError(f"Error: Expected 4 points, but only {len(corners)} were s

    # 4. Compute homography with explicit float32 data types
    pts_src = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]], dtype=np.f
    pts_dst = np.array(corners, dtype=np.float32)

    H, status = cv.findHomography(pts_src, pts_dst)
    if H is None:
        raise RuntimeError("Error: Failed to compute homography matrix from the sel

    # 5. Warp and blend images
    im_out = cv.warpPerspective(im2, H, (im1.shape[1], im1.shape[0]))
    mask = cv.warpPerspective(np.full((h, w), 255, np.uint8), H, (im1.shape[1], im1
    mask3 = (mask > 0)[..., None]

    blended = np.where(
        mask3,
        cv.addWeighted(im1, 1 - alpha, im_out, alpha, 0),
        im1.astype(np.uint8)
    )

    # 6. Display final blended result via OpenCV
    cv.imshow('Blended Image', blended)
    cv.waitKey(0)
    cv.destroyAllWindows()

    # 7. Plot results using Matplotlib
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    titles = ['Original Image', 'Overlay Image', 'Blended Image']
    images = [im1, im2, blended]

    for ax, im, title in zip(axes, images, titles):
        ax.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
        ax.set_title(title)
        ax.axis('off')

    plt.tight_layout()
    plt.show()

    return blended
```

In [ ]:

```
result_image = run_homography_overlay(
    im1_path='images/wadham.jpg',
```

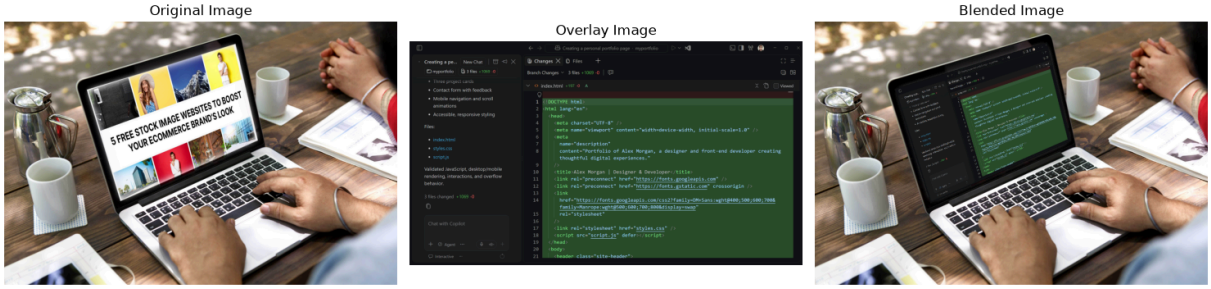
```
    im2_path='images/flag.png'
)
```

```
Image downsampled from 1024x768 to 960x720 to fit HD.
Please click exactly 4 corners on the image window (press 'q' to abort)...
```



In [47]:
```
result_image = run_homography_overlay(
    im1_path='images/Free-Stock-Image-Websites.jpg',
    im2_path='images/changes-panel.png'
)
```

```
Image downsampled from 5000x3337 to 1078x720 to fit HD.
Please click exactly 4 corners on the image window (press 'q' to abort)...
```



Question 4 - SIFT

In [69]:
```
from pathlib import Path
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

# Define the directory containing the PCB images
directory_path = Path("images/pcbs")

# Load the reference PCB image in grayscale and get its dimensions
ref_im = cv.imread(str(directory_path / "AECC1.jpg"), cv.IMREAD_GRAYSCALE)
h, w = ref_im.shape

# Initialize SIFT detector and Brute-Force Matcher (L2 norm for SIFT descriptors)
detector = cv.SIFT_create()
feature_matcher = cv.BFMatcher(cv.NORM_L2)

# Detect keypoints and compute descriptors for the reference image
ref_kpts, ref_desc = detector.detectAndCompute(ref_im, None)
print(f"Reference keypoints: {len(ref_kpts)}")

# List of target PCB images to align and compare against the reference
```

```python
target_images = ["AECC2.jpg", "AAAA1.jpg", "ADAD1.jpg", "AECC3.jpg", "AECC4.jpg"]

for im in target_images:
    # Load the current target image in grayscale
    curr_im = cv.imread(str(directory_path / im), cv.IMREAD_GRAYSCALE)
    kpts, desc = detector.detectAndCompute(curr_im, None)

    # Perform k-NN matching (k=2) to find the two closest matches for each descript
    knn_pairs = feature_matcher.knnMatch(desc, ref_desc, k=2)

    # Apply Lowe's ratio test to filter out ambiguous (weak) matches
    filtered_matches = [m for m, n in knn_pairs if m.distance < 0.75 * n.distance]

    # Ensure unique keypoint mappings (keep only the best match per train index)
    unique_matches_map = {}
    for match_item in filtered_matches:
        existing_match = unique_matches_map.get(match_item.trainIdx)
        if existing_match is None or match_item.distance < existing_match.distance:
            unique_matches_map[match_item.trainIdx] = match_item

    filtered_matches = list(unique_matches_map.values())

    # Extract source and destination coordinates for homography estimation
    source_pts = np.float32([
        kpts[m.queryIdx].pt for m in filtered_matches
    ]).reshape(-1, 1, 2)

    destination_pts = np.float32([
        ref_kpts[m.trainIdx].pt for m in filtered_matches
    ]).reshape(-1, 1, 2)

    # Estimate the homography matrix using RANSAC to handle outliers
    homography_matrix, inliers = cv.findHomography(source_pts, destination_pts, cv.

    # Warp the target image to align it with the reference image coordinate space
    warped_im = cv.warpPerspective(curr_im, homography_matrix, (w, h))

    # Create a validity mask to exclude borders or out-of-bounds warping artifacts
    mask_coverage = cv.warpPerspective(
        np.ones(curr_im.shape, dtype=np.float32), homography_matrix, (w, h)
    )
    valid_mask = mask_coverage > 0.999

    # Compute absolute difference and Mean Absolute Error (MAE) within valid region
    diff_im = cv.absdiff(ref_im, warped_im)
    mean_abs_error = diff_im[valid_mask].mean()

    # Extract and sort inlier matches for visualization
    inlier_matches = [
        m for m, is_inlier in zip(filtered_matches, inliers.ravel()) if is_inlier
    ]
    inlier_matches.sort(key=lambda m: m.distance)

    # Visualize top inlier feature matches between target and reference images
    match_viz = cv.drawMatches(
        curr_im, kpts,
```

```
        ref_im, ref_kpts,
        inlier_matches[:80], None,
        matchColor=(0, 255, 0),
        flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    plt.figure(figsize=(14, 6))
    plt.imshow(cv.cvtColor(match_viz, cv.COLOR_BGR2RGB))
    plt.title(f"{im} → AECC1.jpg: inlier matches (up to 80 shown)")
    plt.axis("off")
    plt.tight_layout()
    plt.show()

    # Display a side-by-side comparison: Reference, Aligned Image, and Difference M
    fig, subplots = plt.subplots(1, 3, figsize=(15, 5))

    display_outputs = [
        ref_im,
        warped_im,
        np.ma.array(diff_im, mask=~valid_mask)
    ]
    display_titles = ["Reference", f"Aligned: {im}", "Absolute difference"]

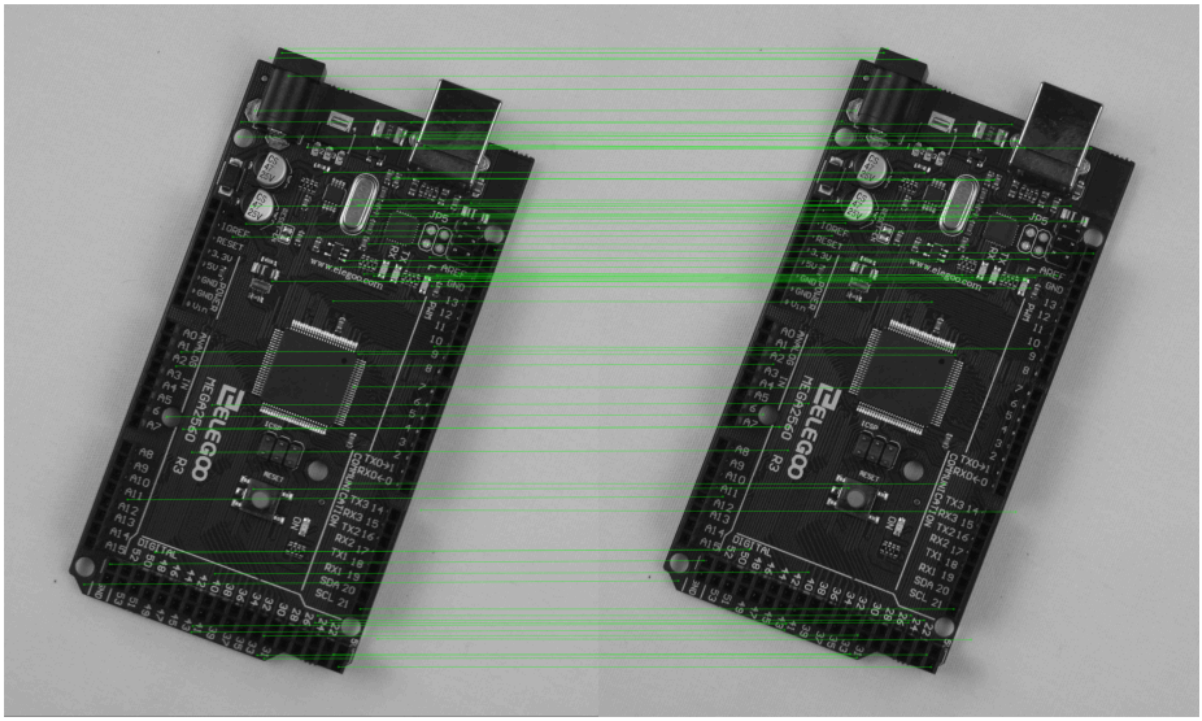
    for ax, out_im, title_text in zip(subplots, display_outputs, display_titles):
        ax.imshow(out_im, cmap="gray", vmin=0, vmax=255)
        ax.set_title(title_text)
        ax.axis("off")

    plt.tight_layout()
    plt.show()

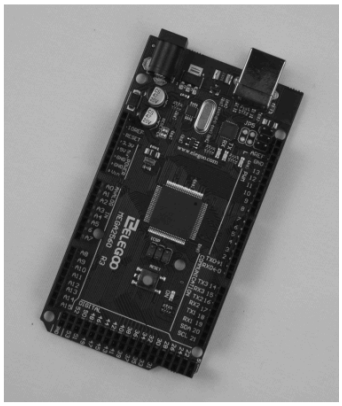
    # Print summary metrics for the current target image
    print(
        f"{im}: keypoints={len(kpts)}, "
        f"unique matches={len(filtered_matches)}, "
        f"inliers={len(inlier_matches)}, "
        f"overlap={100 * valid_mask.mean():.1f}%, "
        f"MAE={mean_abs_error:.2f}"
    )
```

```
Reference keypoints: 14271
```

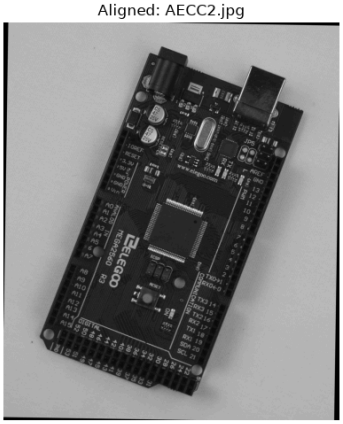
AECC2.jpg → AECC1.jpg: inlier matches (up to 80 shown)



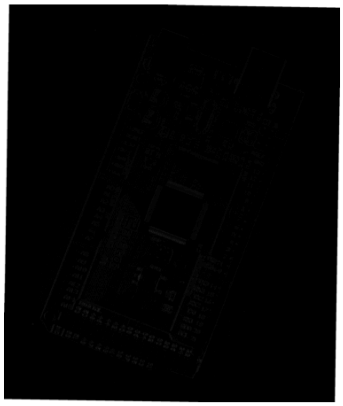
Reference



Aligned: AECC2.jpg

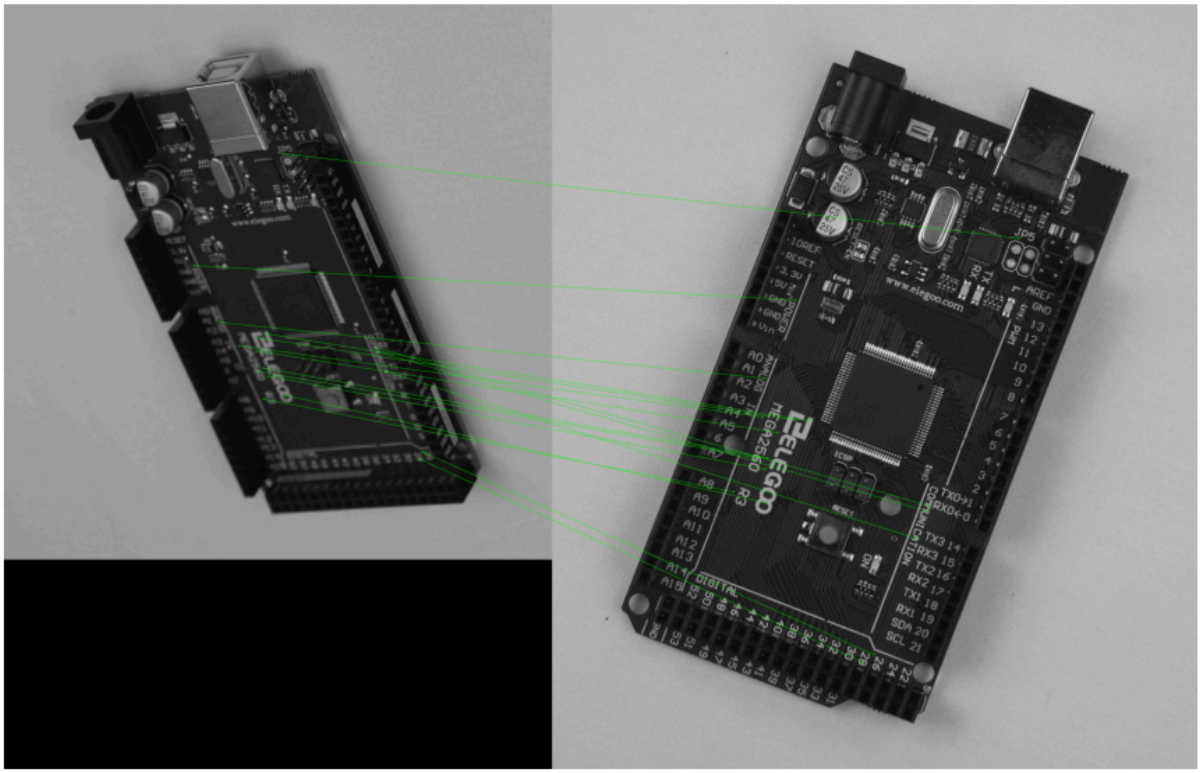


Absolute difference

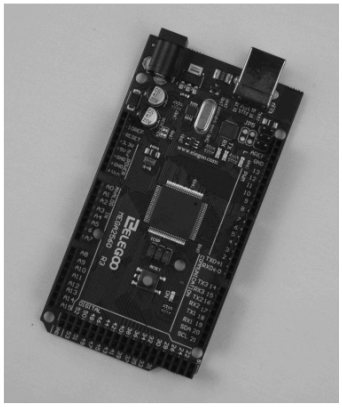


AECC2.jpg: keypoints=12312, unique matches=4110, inliers=3805, overlap=98.4%, MAE=3.00

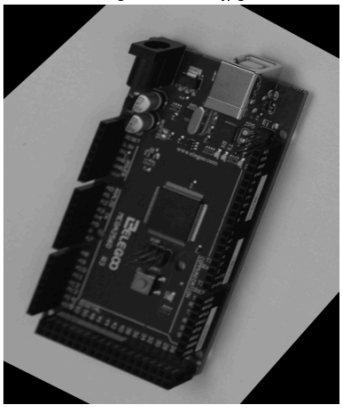
AAAA1.jpg → AECC1.jpg: inlier matches (up to 80 shown)



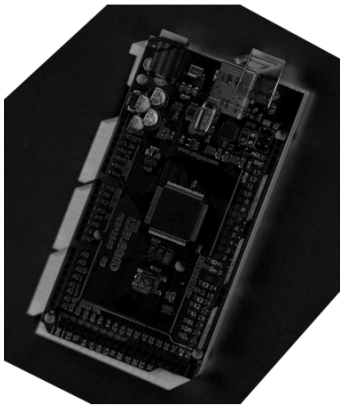
Reference



Aligned: AAAA1.jpg

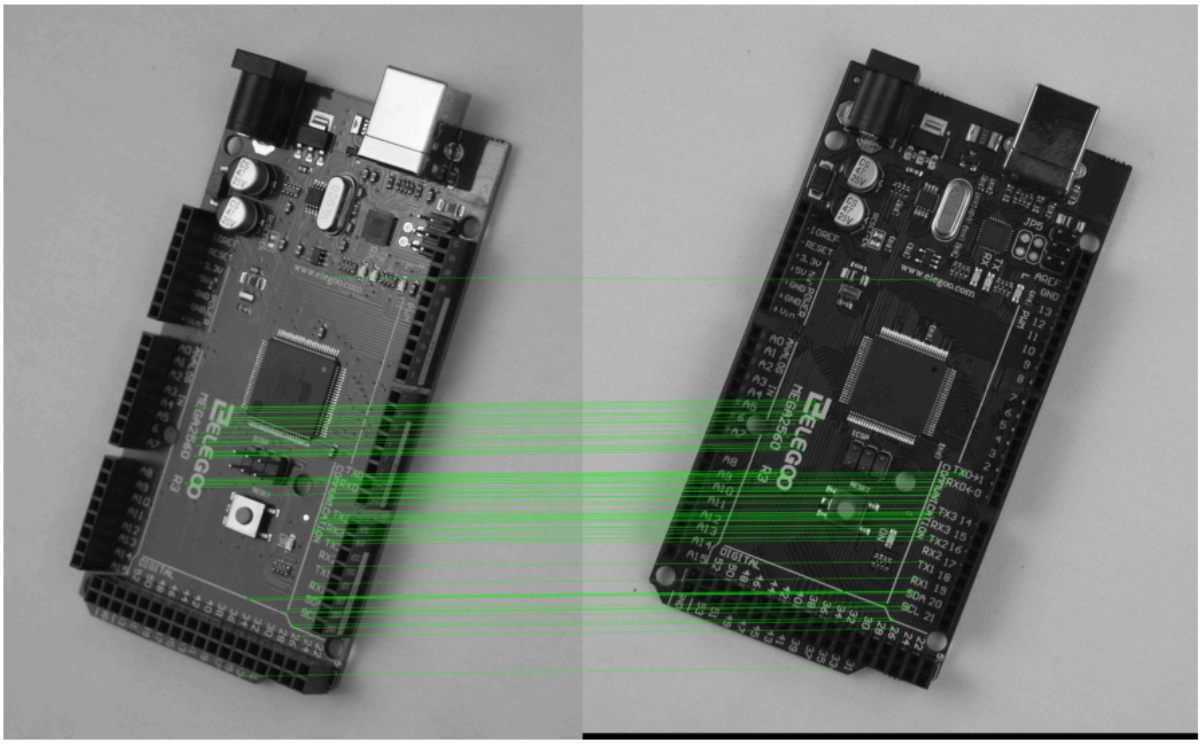


Absolute difference

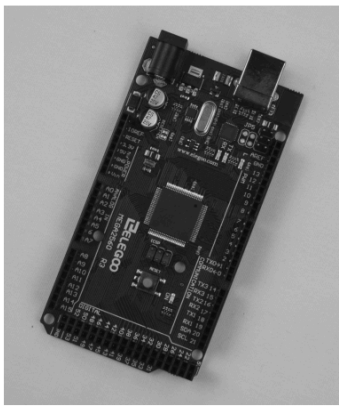


AAAA1.jpg: keypoints=2413, unique matches=47, inliers=17, overlap=88.8%, MAE=28.32

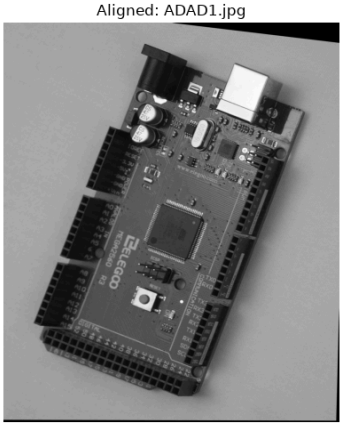
ADAD1.jpg → AECC1.jpg: inlier matches (up to 80 shown)



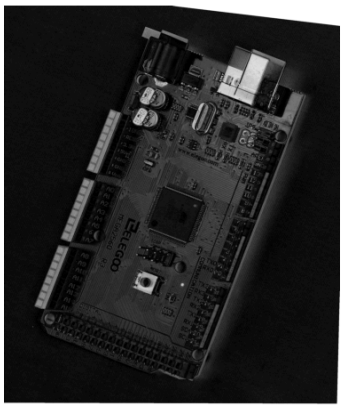
Reference



Aligned: ADAD1.jpg

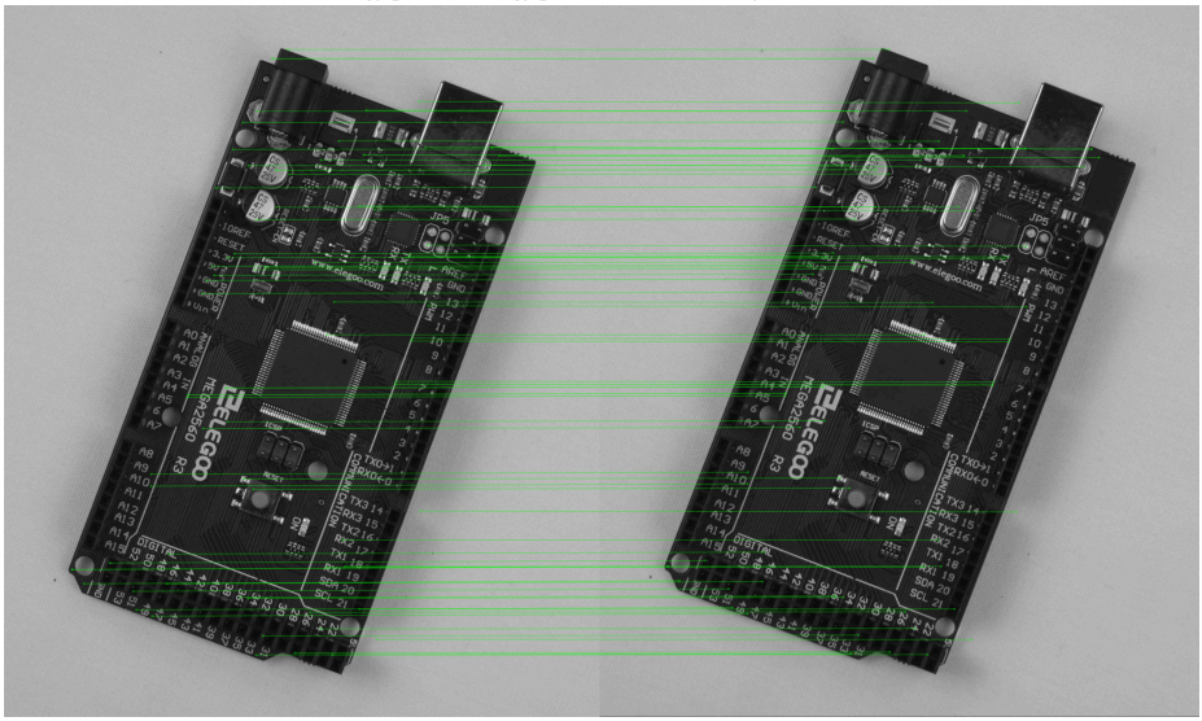


Absolute difference

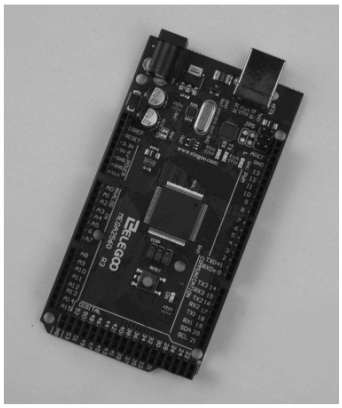


ADAD1.jpg: keypoints=8488, unique matches=300, inliers=190, overlap=98.1%, MAE=31.16

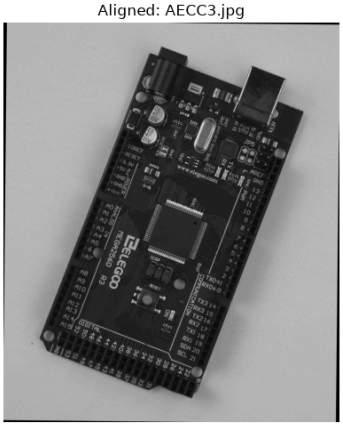
AECC3.jpg → AECC1.jpg: inlier matches (up to 80 shown)



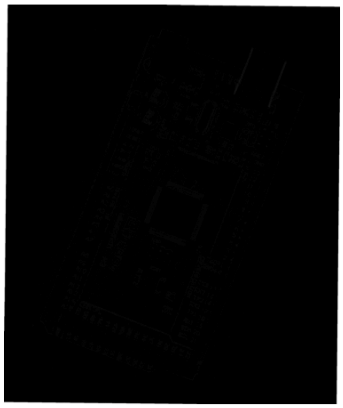
Reference



Aligned: AECC3.jpg

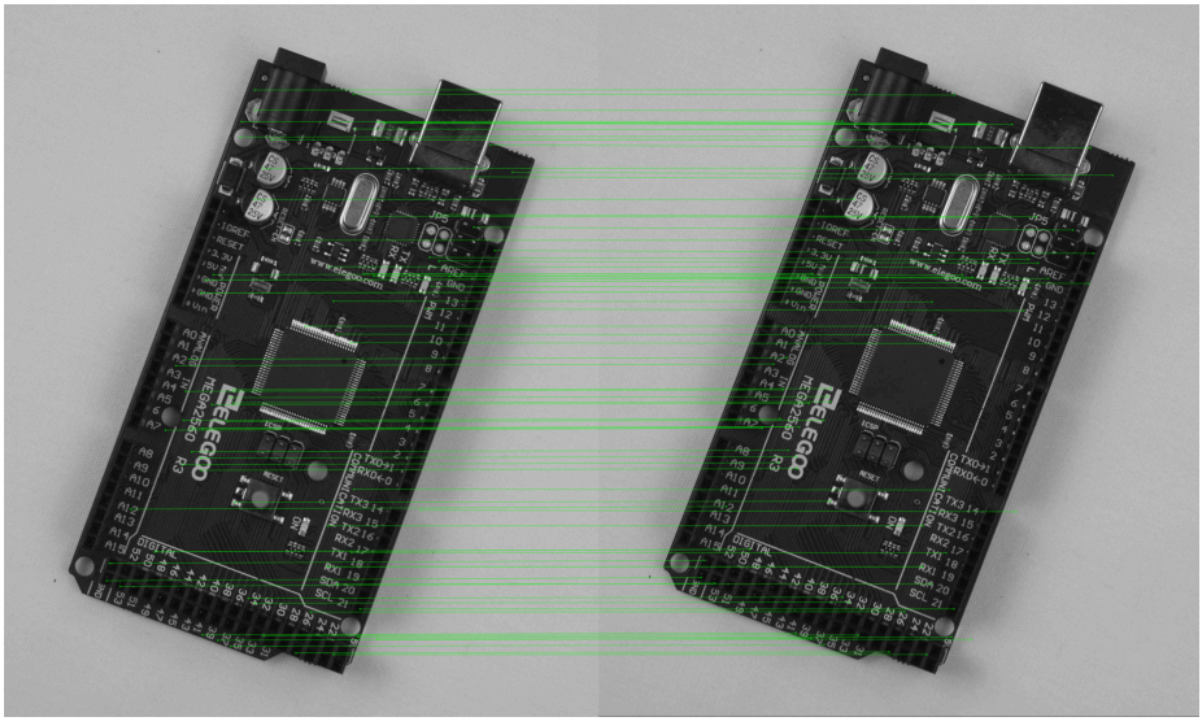


Absolute difference

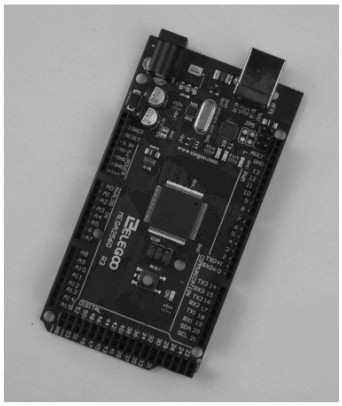


AECC3.jpg: keypoints=15197, unique matches=5332, inliers=4934, overlap=98.9%, MAE=2.78

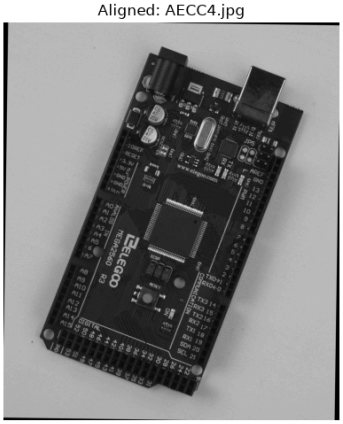
AECC4.jpg → AECC1.jpg: inlier matches (up to 80 shown)



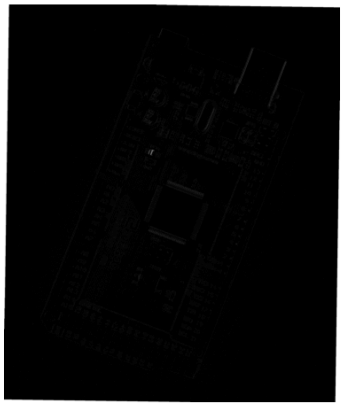
Reference



Aligned: AECC4.jpg



Absolute difference



AECC4.jpg: keypoints=11610, unique matches=3602, inliers=3342, overlap=98.5%, MAE=3.58